

Cloudfactory Inference Infrastructure

This guide provides additional insight on the architecture diagram and Whimsical drawing guide. This is a proposed design. The examples describe the intended architecture and do not deploy a working platform.

Running the Models

Trained model artifacts are stored in Amazon S3. Each model release uses a specific, pinned artifact so that the exact model version can be reproduced. S3 acts as the durable source of the trained model, but the serving platform does not download the model from S3 for every prediction.

GitHub Actions handles the model packaging and image-build process. During the CI/CD pipeline, it retrieves the pinned model artifact from S3, validates it, and bundles it together with the model-serving code and dependencies into a Docker image. The resulting image represents the complete deployable artifact for that model version.

The Docker image is pushed to Amazon ECR and identified by an immutable image digest. This means the deployed image contains the exact model, serving code, and dependencies used for that release. Kubernetes nodes pull this image when starting model pods.

EKS/Kubernetes runs the model containers as independently scalable pods. Each model version has its own Deployment, so scaling or updating them are independent. Models load into memory when the pod starts and remain stateless between requests.

How Request Travels

The request flow is Client → DNS → NLB → Traefik → selected model-version pod → response. DNS resolves the hostname, and the AWS Network Load Balancer distributes TCP connections to the Traefik replicas. Traefik then handles the HTTP-level routing and selects the appropriate model version.

The AWS Load Balancer Controller watches the Kubernetes configuration and creates or updates the AWS load balancer associated with the gateway Service. This keeps the AWS load-balancing infrastructure aligned with the Kubernetes configuration.

Traefik acts as the ingress controller and gateway. It terminates TLS, applies middleware, performs routing, and selects the appropriate backend. It does not host the models or scale model replicas.

Scaling

The **Horizontal Pod Autoscaler (HPA)** uses the selected metric and a benchmarked per-replica target to calculate the required number of model replicas. It keeps the number within the configured minimum and maximum limits.

The Kubernetes **Deployment** maintains the replica count requested by the HPA. When the target increases, Kubernetes creates additional model pods. When demand falls, it can remove replicas according to the configured scaling and stabilization policies.

Karpenter handles the next level of scaling. If the required pods cannot fit on the existing worker nodes, Karpenter provisions additional machines with the appropriate capacity. Critical Kubernetes controllers remain on managed on-demand system nodes so the platform does not depend on Karpenter to create the infrastructure required to operate Karpenter itself.

Deployment Infrastructure

Terraform defines the AWS infrastructure as code. `plan` previews infrastructure changes, while `apply` executes them. Terraform state is stored in encrypted and versioned S3, with locking used to prevent competing infrastructure changes.

Helm provides a shared Kubernetes chart so that every model version follows the same deployment structure. Shared environment settings can be combined with a separate values file for each model version, while different release names keep the deployments independent.

Argo CD provides the GitOps deployment mechanism. It continuously reconciles the Kubernetes cluster with the configuration stored in Git. Application configuration is managed by GitOps while the HPA remains responsible for the runtime replica count.

GitHub Actions performs verification, testing, model packaging, Docker image builds, and promotion workflows. It uses OIDC to obtain temporary AWS credentials rather than relying on long-lived AWS access keys. The pipeline retrieves the pinned model artifact from S3, packages it into the Docker image, tests the image, and pushes it to ECR.

Reliability and Security

Pod placement and PodDisruptionBudgets help keep replicas distributed across nodes and availability zones. A PDB limits voluntary disruptions but cannot protect against an actual machine or availability-zone failure.

IAM and workload identities limit what each workload can access in AWS. Pods receive only the permissions they require, while sensitive configuration and credentials are retrieved from a managed secrets store rather than being embedded directly in images or configuration files.

RBAC and NetworkPolicies provide Kubernetes-level security. RBAC controls access to the Kubernetes API, while NetworkPolicies restrict communication between pods when supported and enforced by the networking implementation. The CNI provides the underlying pod networking.

For **image security**, container images should be verified using signatures or digests, dependencies should be scanned for vulnerabilities, and a software bill of materials (SBOM) should be retained. Containers should also run with only the privileges they require.

Observability

Prometheus collects operational metrics such as request rate, errors, latency, replica count, and CPU/GPU utilisation. Multiple Prometheus replicas can improve monitoring availability, while the scaling configuration should use a single consistent metric source to avoid double-counting.

Grafana provides dashboards for Kubernetes, infrastructure, the gateway, model-serving services, and capacity. It gives engineers a visual view of the system and can link operational metrics with logs and traces.

Datadog provides broader monitoring, alerting, service visibility, and on-call capabilities. It can bring together infrastructure and application signals and provide operational alerts that help engineers respond to incidents.

CloudWatch Logs provides AWS and application log visibility. Structured log events and request IDs help engineers understand failures and follow a request through the system. Raw inference payloads are not logged by default to avoid unnecessarily storing sensitive data.

OpenTelemetry provides distributed tracing. Traces show where time is spent as a request moves through the gateway, authentication service, and model-serving components. Where the model container is opaque, its HTTP duration can still be captured even if its internal implementation is not instrumented.

The proposed **SLOs** are 99.9% eligible-request success and a p95 latency of 300 ms, meaning 95% of successful requests should complete within that time. Alerts should track the remaining error budget, while synthetic requests exercise the complete production path rather than checking individual components in isolation.

Conclusion

The architecture trades some operational complexity and infrastructure cost for flexible placement, independent model-version scaling, controlled releases, and resilience.

Kubernetes provides independent scaling and flexible workload placement, while Karpenter allows worker capacity to grow when additional model replicas cannot fit. Keeping warm replicas and rollback capacity increases cost, but reduces cold-start and recovery risk. Spot capacity can reduce costs for suitable workloads, while on-demand capacity provides the predictable baseline required for critical inference.